

Aluno: Luis Eduardo Bonfim Bassi
Curso: Engenharia de Software
Disciplina: Árvores e Grafos
Professor: Prof. Vinícius Siqueira
Campus: Anápolis
Data: 25/02/26

RELATORIO TAREFA DO VINICIUS

1- Como os Algoritmos Foram Implementados

Todos os algoritmos foram implementados em funções separadas dentro de um único arquivo tarefa arvores e grafos 2.c, conforme permitido pelas instruções da atividade.

Foi criada uma função auxiliar:

`printArray()` → utilizada para imprimir o vetor antes e depois da ordenação.

Também foram criadas funções auxiliares para:

Gerar vetor em ordem decrescente (pior caso)

Copiar o vetor original, garantindo que todos os algoritmos utilizassem o mesmo conjunto de dados

A medição de desempenho foi realizada da seguinte forma:

```
clock_t inicio = clock();  
algoritmo(vetor, tamanho);  
clock_t fim = clock();
```

```
double tempo = (double)(fim - inicio) / CLOCKS_PER_SEC;
```

O programa possui um menu interativo, permitindo ao usuário escolher qual algoritmo deseja executar.

1.1 Bubble Sort

Implementado utilizando dois loops aninhados, comparando elementos adjacentes e realizando trocas quando necessário.

Complexidade:

Melhor caso: $O(n)$

Pior caso: $O(n^2)$

1.2 Selection Sort

Implementado buscando o menor elemento da parte não ordenada e trocando com a posição atual.

Complexidade:

$O(n^2)$ em todos os casos

1.3 Insertion Sort

Implementado inserindo cada elemento na posição correta dentro da parte já ordenada do vetor.

Complexidade:

Melhor caso: $O(n)$

Pior caso: $O(n^2)$

1.4 Merge Sort

Implementado utilizando divisão e conquista, com função auxiliar merge() para mesclar subvetores.

Complexidade:

$O(n \log n)$

Utiliza memória adicional $O(n)$

1.5 Quick Sort

Implementado utilizando particionamento com pivô e chamadas recursivas.

Complexidade:

Média: $O(n \log n)$

Pior caso: $O(n^2)$

1.6 Shell Sort

Implementado utilizando gaps decrescentes, reduzindo gradualmente até 1.

Complexidade:

Variável

Pode chegar a $O(n^2)$

2. Resultados Obtidos nos Testes

Os testes foram realizados utilizando vetores em ordem decrescente, representando o pior caso para a maioria dos algoritmos.

Foram utilizados diferentes tamanhos de entrada, como:

- 1.000 elementos
- 5.000 elementos
- 10.000 elementos

Observações gerais:

- Bubble Sort apresentou maior tempo de execução.
- Selection Sort apresentou desempenho semelhante ao Bubble, porém com menos trocas.
- Insertion Sort teve bom desempenho para tamanhos menores.
- Merge Sort apresentou excelente desempenho para grandes volumes.
- Quick Sort foi um dos mais rápidos.
- Shell Sort apresentou desempenho intermediário.

Os resultados confirmaram a teoria da notação Big-O estudada em aula

3. Comparação de Desempenho Entre os Algoritmos

Algoritmo	Complexidade	Estável	Uso de Memória	Desempenho em Grandes Volumes
Bubble Sort	$O(n^2)$	Sim	Baixo	Ruim
Selection Sort	$O(n^2)$	Não	Baixo	Ruim
Insertion Sort	$O(n^2)$	Sim	Baixo	Médio (listas pequenas)
Merge Sort	$O(n \log n)$	Sim	Alto	Excelente
Quick Sort	$O(n \log n)^*$	Não	Médio	Excelente
Shell Sort	Variável	Não	Baixo	Bom

A análise mostra que algoritmos quadráticos tornam-se inviáveis para grandes volumes de dados, enquanto algoritmos $O(n \log n)$ são mais adequados para aplicações reais.